

Addition Of Two Integers

—Sum Things Never Change

Protyasha Kundu

July 2026

1 Introduction

Remember how we used to count the candies we collected at birthday parties?

“I have three.” “I have seven.” “I have thirteen.”- total victory.

Whoever had the biggest number felt like the winner. The art of adding things is inherent to humans since childhood.

But why do we spend so much time learning to add numbers? What purpose does it serve?

The answer is simple: addition of integers is one of the fundamental building blocks of mathematics. Almost every branch of mathematics, from arithmetic and algebra to calculus and computer science, relies on the ability to combine quantities. More complex operations such as multiplication, exponentiation, and many algorithms are built upon the simple idea of addition. By understanding addition, we lay the foundation for nearly everything that follows in mathematics.

2 Art of Addition

Lets go back to the days of elementary school - how we have learned to add. The numbers are written one below the other, neatly aligned by their place values. Starting from the rightmost digit, the digits are added one by one. Whenever the sum becomes larger than nine, the extra value is quietly “carried” over to the next column. Step by step, what once seemed like a difficult task turns into a simple and elegant routine.

At first, it feels like nothing more than a classroom exercise. Yet hidden within this familiar method is a remarkably powerful algorithm. Whether adding two small numbers or two integers with hundreds of digits, the same sequence of steps always works. The beauty of this process lies in its simplicity—every digit is treated independently, carries are propagated only when needed, and a complex calculation is broken into a series of easy operations.

Years later, when we open a computer or use a smartphone, we discover something fascinating. Although computers know nothing about carrying digits on paper or writing numbers in columns, they perform addition using the very

same underlying idea. Instead of pencils and notebooks, they rely on logic gates and electronic circuits. The elegant method we first encountered in primary school becomes the inspiration for the arithmetic circuits that form the heart of every modern processor.

2.1 Full Adder Circuit

A full adder is a combinational logic circuit that performs the addition of three binary inputs: two operand bits (A and B) and an incoming carry (C_{in}) from the previous stage. It produces two outputs: the Sum (SUM) and the Carry-out (C_{out}).

A	B	C_{in}	SUM	$Carry_{out}$
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Truth Table of a Full Adder

$$SUM = A \oplus B \oplus C_{in}, \quad (1)$$

$$Carry_{out} = AB + AC_{in} + BC_{in}. \quad (2)$$

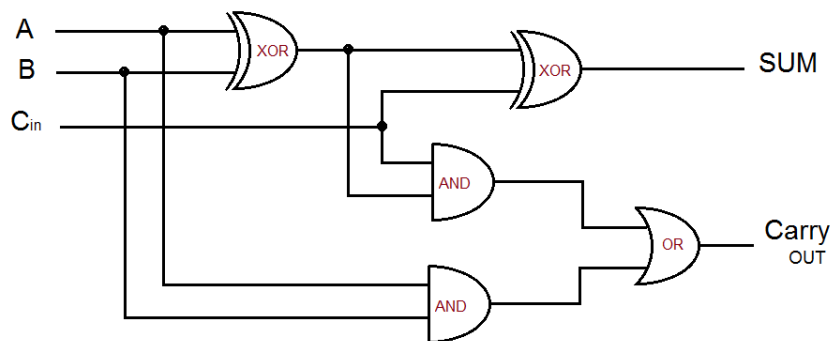


Figure 1: Full Adder Circuit

2.2 n -bit Ripple Carry Adder/Subtractor

An n -bit ripple carry adder/subtractor is a combinational arithmetic circuit constructed by cascading n full adders to perform addition and subtraction of two n -bit binary numbers. The operation is controlled by a single mode input, M . Each bit of the second operand, B_i , is XORed with the mode signal before being applied to the full adder. When $M = 0$, the XOR gates pass the bits of B unchanged, and the circuit performs binary addition, i.e., $A + B$. When $M = 1$, each bit of B is complemented by the XOR gates, and the initial carry-in is also set to 1, thereby forming the two's complement of B . Consequently, the circuit performs subtraction as $A + \overline{B} + 1 = A - B$. This design eliminates the need for separate hardware for addition and subtraction, making the n -bit full adder/subtractor an efficient and widely used arithmetic unit in modern digital systems.

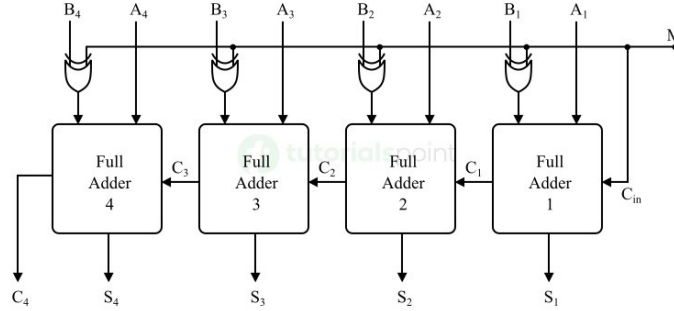


Figure 2: n -bit Ripple Carry Adder/Subtractor

$$F = \begin{cases} A + B, & \text{if } M = 0, \\ A + \overline{B} + 1, & \text{if } M = 1 \quad (= A - B). \end{cases} \quad (3)$$

For the i^{th} bit, the Boolean equations are

$$B'_i = B_i \oplus M, \quad (4)$$

$$S_i = A_i \oplus B'_i \oplus C_i, \quad (5)$$

$$C_{i+1} = A_i B'_i + A_i C_i + B'_i C_i, \quad (6)$$

where

- A_i and B_i denote the i^{th} bits of the input operands,
- B'_i is the modified second operand,
- C_i is the carry input to the i^{th} full adder,

- C_{i+1} is the carry output from the i^{th} full adder, and
- M is the mode control signal, where $M = 0$ selects addition and $M = 1$ selects subtraction.

The operation of the n -bit ripple carry adder/subtractor is summarized

Mode (M)	Modified Operand	Initial Carry	Operation
0	B_i	0	$A + B$
1	$\overline{B_i}$	1	$A + \overline{B} + 1 = A - B$

Table 2: Operation of an n -bit Full Adder/Subtractor

3 Conclusion

What once seemed like a simple classroom exercise turns out to be the very foundation of digital computation. Beneath every calculation performed by a computer lies an elegant network of logic gates working together to mimic the familiar process of carrying digits from one column to the next. The full adder and the n -bit adder/subtractor demonstrate how a handful of Boolean operations can implement one of the most fundamental mathematical tasks with remarkable efficiency. It is a gentle reminder that sometimes the most sophisticated technologies are built upon the simplest ideas we learned as children. After all, every billion-dollar processor is, at its heart, still just adding numbers—only a few billion times every second.

4 Reference

Images from Tutorials Point